

SICAE

Sistema de Control de Acceso de Estacionamiento

Ruta de Trabajo Estratégica para Obtener el 100%

Proyecto Final - Tecnologías para la Integración de Soluciones

Propósito del documento

Organizar la ejecución del proyecto de la forma más sencilla, rápida y eficiente posible, enfocándose únicamente en lo que suma puntos: microservicios funcionales, seguridad JWT, bases independientes, contenedores, Postman y documentación clara.

Enfoque	Decisión estratégica
Qué sí importa	Funcionamiento real de APIs, reglas de negocio, JWT, contenedores, pruebas y documentación.
Qué no se hará	No se desarrollará cliente web, app móvil, app de escritorio ni interfaz gráfica.
Prioridad	Demostrar el flujo completo: login, token, usuarios, vehículos, entrada, salida y evidencias.
Forma de trabajo	Cadena por fases: cada parte debe dejar insumos listos para la siguiente.

Contenido de la ruta

1. Diagnóstico estratégico del proyecto.
2. Puntos críticos de éxito para la calificación perfecta.
3. Ruta de trabajo paso a paso.
4. Delegación general para 4 integrantes.
5. Alertas de complicaciones y cómo evitarlas.
6. Checklist final antes de entregar y exponer.

1. Diagnóstico estratégico

Idea central

El proyecto no se gana haciendo una aplicación visualmente atractiva; se gana demostrando microservicios REST funcionando, con autenticación, autorización, bases separadas, contenedores, pruebas y documentación verificable.

Elemento del proyecto	Lectura eficiente
Arquitectura de microservicios	Separar responsabilidades por dominio y poder explicar independencia, despliegue y comunicación.
Arquitectura por capas	Cada servicio debe organizarse internamente en Controller, Service y Repository.
Seguridad	Login con JWT y contraseñas cifradas con bcrypt. El token debe usarse en los demás servicios.
Persistencia distribuida	Base independiente por dominio: usuarios en PostgreSQL, vehículos en MySQL y estacionamiento en MySQL.
Contenerización	Cada microservicio y base debe poder levantarse en contenedor.
Pruebas	Postman debe demostrar casos exitosos y casos de error.
Documentación	Debe seguir la plantilla y tener diagramas, endpoints, evidencias y conclusiones.

2. Puntos críticos de éxito

- El sistema debe funcionar correctamente durante la demostración: esta es la prioridad número uno.
- Deben existir servicios claros para autenticación, usuarios, vehículos y estacionamiento.
- El flujo JWT debe verse en Postman: login, obtención de token y consumo de servicios protegidos.
- Cada microservicio debe mostrar estructura Controller, Service y Repository.
- Las bases de datos deben estar separadas por dominio y no depender de relaciones físicas entre dominios.
- Las reglas de negocio principales deben poder demostrarse con casos exitosos y casos de error.
- La colección de Postman debe estar ordenada como una demo guiada, no como requests sueltos.
- Docker debe estar listo antes del último día para evitar fallos por puertos, variables o conexiones.
- La documentación debe ser clara, con tablas de endpoints, diagramas y capturas reales.
- Todos los integrantes deben poder explicar una parte técnica del sistema.

Regla de oro

No se considera terminada ninguna fase si no tiene: funcionalidad probada, evidencia, explicación corta y entregable útil para la siguiente fase.

3. Ruta de trabajo estructurada

Fase 1. Congelar el alcance

Objetivo de la fase

Definir exactamente qué se construirá y qué se ignorará para no perder tiempo en complejidad que no suma puntos.

Acciones esenciales:

- Confirmar que no habrá interfaz gráfica, cliente web, aplicación móvil ni app de escritorio.
- Definir los servicios: AuthService, UserService, VehicleService y ParkingService.
- Acordar puertos, nombres de carpetas, estilo de endpoints y estructura de paquetes.
- Acordar una rama o repositorio de trabajo y reglas básicas para integrar avances.

Entregables esperados:

- Estructura general del proyecto lista.
- Lista de servicios, puertos y responsables.
- Decisiones técnicas congeladas para evitar cambios innecesarios.

Criterio de cierre

Todos deben saber qué servicio desarrollan, qué base usa y qué endpoint entrega al siguiente integrante.

Evitar para no perder tiempo

No agregar tecnologías extra como frontend, Kubernetes, API Gateway o paneles visuales si no son exigidos.

Fase 2. Preparar bases de datos y datos de prueba

Objetivo de la fase

Importar los scripts entregados y preparar datos mínimos para probar los flujos principales y los errores importantes.

Acciones esenciales:

- Importar PostgreSQL para usuarios, roles, tipos de usuario y programas educativos.
- Importar MySQL para marcas, modelos y vehículos.
- Importar MySQL para cajones y movimientos de estacionamiento.
- Crear o verificar usuarios activos, usuarios inactivos, vehículos activos, vehículos inactivos y cajones libres.
- Guardar credenciales y datos de prueba en una hoja común del equipo.

Entregables esperados:

- Tres bases funcionando.
- Datos de prueba definidos para Postman.
- Capturas de las bases cargadas.
- Tabla de usuarios, placas, cajones y credenciales de prueba.

Criterio de cierre

El equipo debe poder probar login, registro, placas repetidas, límites de vehículos, entrada y salida sin inventar datos durante la demo.

Fase 3. Construir AuthService

Objetivo de la fase

Crear primero la autenticación porque todos los demás servicios dependen del token.

Acciones esenciales:

- Implementar login con usuario y contraseña.
- Validar campos obligatorios, existencia del usuario, contraseña correcta y usuario activo.
- Generar JWT con los datos necesarios del usuario.
- Validar contraseñas cifradas con bcrypt.
- Probar login exitoso, login incorrecto y usuario inactivo.

Entregables esperados:

- AuthService funcional.
- Endpoint de login probado en Postman.
- Token válido para consumir servicios protegidos.
- Capturas de login exitoso y errores.
- Explicación breve del uso de JWT y bcrypt.

Criterio de cierre

Sin AuthService terminado no se debe avanzar a cerrar los demás servicios, porque la seguridad es una regla transversal.

Fase 4. Construir UserService

Objetivo de la fase

Implementar la administración de usuarios y dejar un endpoint de validación que pueda consumir ParkingService.

Acciones esenciales:

- Registrar usuarios nuevos solo con rol administrador.
- Validar correo, username, campos obligatorios y tamaños de campo.
- Evitar usuarios duplicados por correo o username.
- Cifrar contraseña al registrar.
- Generar clave única de usuario.
- Editar usuario sin permitir cambios directos de usuario, contraseña ni clave.
- Consultar perfil completo.
- Cambiar estatus activo/inactivo sin eliminación física.
- Crear validación de usuario por clave para uso de ParkingService.

Entregables esperados:

- UserService funcional.
- Endpoints de registro, edición, perfil y estatus.
- Casos exitosos y de error en Postman.
- Endpoint estable para validar usuario activo.
- Capturas y descripción para documentación.

Criterio de cierre

Debe quedar disponible el idUsuario y la claveUsuario de prueba para que vehículos y estacionamiento

puedan continuar.

Fase 5. Construir VehicleService

Objetivo de la fase

Administrar vehículos por usuario y dejar validaciones listas para el servicio de estacionamiento.

Acciones esenciales:

- Buscar vehículos asociados a un usuario específico.
- Registrar vehículo asociado a usuario.
- Validar placa única.
- Asignar vehículo activo por defecto.
- Validar máximo 4 vehículos activos por usuario.
- Editar solo vehículos asociados al usuario autenticado.
- Cambiar estatus activo/inactivo sin eliminar físicamente.
- Crear validación de placa e idUsuario para ParkingService.

Entregables esperados:

- VehicleService funcional.
- Endpoints de búsqueda, registro, edición y estatus.
- Casos exitosos y de error en Postman.
- Endpoint estable para validar placa, estatus y pertenencia.
- Capturas y descripción para documentación.

Criterio de cierre

Debe quedar una placa activa asociada a un usuario activo para que ParkingService pueda probar entrada y salida.

Fase 6. Construir ParkingService

Objetivo de la fase

Implementar el flujo central del estacionamiento: consultar espacios, registrar entrada, registrar salida, calcular tiempo y costo.

Acciones esenciales:

- Consultar espacios disponibles.
- Registrar entrada con claveUsuario, placa, hora de entrada, tarifa y espacio.
- Validar usuario activo mediante UserService.
- Validar vehículo activo y asociado mediante VehicleService.
- Validar máximo 2 vehículos dentro por usuario.
- Ocupar el cajón cuando se registre la entrada.
- Registrar salida con cálculo de minutos, horas cobradas y costo total.
- Liberar el cajón al registrar la salida.

Entregables esperados:

- ParkingService funcional.
- Entrada exitosa y salida exitosa.
- Consulta de espacios antes y después de entrada/salida.
- Errores probados: usuario inactivo, vehículo no asociado, tercer vehículo y cajón ocupado.

- Capturas del flujo completo.

Criterio de cierre

Esta fase conecta todo el sistema; si ParkingService funciona, la demo tiene una historia completa y convincente.

Fase 7. Comunicación entre microservicios

Objetivo de la fase

Hacer que ParkingService valide usuario y vehículo mediante REST, sin consultar directamente bases de otros dominios.

Acciones esenciales:

- Definir endpoints internos simples para validar usuario activo y vehículo asociado.
- Probar que ParkingService pueda consumir UserService.
- Probar que ParkingService pueda consumir VehicleService.
- Manejar respuestas de error cuando un servicio no valide la información.
- Documentar el flujo de comunicación entre servicios.

Entregables esperados:

- Validación REST entre servicios funcionando.
- Flujo documentado para entrada y salida.
- Evidencia de que no hay consultas directas entre bases de distintos dominios.

Criterio de cierre

La integración debe ser desacoplada: cada microservicio mantiene su base y expone validaciones por API.

Fase 8. Docker y despliegue

Objetivo de la fase

Contenerizar servicios y bases para cumplir portabilidad, aislamiento y despliegue distribuido.

Acciones esenciales:

- Crear Dockerfile por microservicio.
- Crear configuración para el servidor 1: AuthService, UserService y PostgreSQL.
- Crear configuración para el servidor 2: VehicleService y MySQL de vehículos.
- Crear configuración para el servidor 3: ParkingService y MySQL de estacionamiento.
- Usar variables de entorno para credenciales, puertos y conexiones.
- Probar los endpoints contra los servicios levantados en contenedores.

Entregables esperados:

- Dockerfiles de los microservicios.
- Archivos de configuración para contenedores.
- Bases levantadas en contenedor.
- Captura de contenedores corriendo.
- Prueba de Postman contra contenedores.

Criterio de cierre

Docker debe probarse antes de la documentación final, porque las capturas y evidencias deben salir del entorno real de demostración.

Evitar para no perder tiempo

No usar localhost entre contenedores si eso rompe la comunicación; usar nombres de servicio o configuración de red.

Fase 9. Postman y pruebas funcionales

Objetivo de la fase

Crear una colección ordenada como demo guiada, con casos exitosos y casos de error.

Acciones esenciales:

- Crear variables para URLs base, token, idUsuario, claveUsuario, placa, idVehiculo, idMovimiento e idEspacio.
- Ordenar requests por carpetas: Auth, Usuarios, Vehículos y Estacionamiento.
- Guardar token después de login para reutilizarlo.
- Probar cada regla importante con al menos un caso exitoso y un caso de error.
- Exportar colección final para entregar.

Entregables esperados:

- Colección Postman completa.
- Variables configuradas.
- Casos exitosos y errores preparados.
- Capturas listas para documentación.
- Guion de demo basado en la colección.

Criterio de cierre

La colección debe poder ejecutarse de arriba hacia abajo como una historia: login, usuario, vehículo, espacios, entrada y salida.

Fase 10. Documentación técnica

Objetivo de la fase

Llenar la plantilla con información clara, diagramas, tablas de API y evidencias reales del sistema.

Acciones esenciales:

- Redactar introducción y objetivos alineados al proyecto.
- Explicar cada microservicio y su responsabilidad.
- Incluir arquitectura de microservicios y arquitectura por capas.
- Agregar diagramas: general, despliegue, componentes por capas y flujo JWT.
- Crear tabla de endpoints con método, ruta, descripción, autenticación, request y response.
- Incluir tecnologías de desarrollo, almacenamiento y despliegue.
- Agregar pruebas, casos exitosos, casos de error y evidencias.
- Cerrar con resultados, aprendizajes y problemas encontrados.

Entregables esperados:

- Documento técnico completo.
- Diagramas legibles.
- Tablas de API REST.
- Capturas reales de Postman, contenedores y bases.
- Conclusiones del equipo.

Criterio de cierre

La documentación debe servir para que el profesor pueda revisar previamente y validar la demostración.

Evitar para no perder tiempo

No llenar páginas con teoría genérica; debe verse que el documento describe el sistema real implementado.

Fase 11. Ensayo de demostración

Objetivo de la fase

Practicar la exposición para que todos expliquen su parte y el sistema no falle durante la revisión.

Acciones esenciales:

- Levantar contenedores desde cero.
- Ejecutar la colección Postman en el orden de demo.
- Verificar que el token funcione.
- Probar entrada y salida del estacionamiento.
- Mostrar al menos un error controlado.
- Asignar qué explica cada integrante.
- Preparar respuestas para preguntas sobre arquitectura, seguridad, bases y contenedores.

Entregables esperados:

- Demo ensayada.
- Roles de exposición definidos.
- Checklist final aprobado.
- Paquete final listo para comprimir y entregar.

Criterio de cierre

Todos deben participar y poder explicar tanto su servicio como la relación con el sistema completo.

4. Delegación general para 4 integrantes

Integrante	Responsabilidad principal	Entregables clave
Integrante 1	Arquitectura base, AuthService, JWT, bcrypt y apoyo de Docker inicial.	AuthService funcional, token, login probado, diagrama JWT, estructura del proyecto y configuración base.
Integrante 2	UserService y dominio de usuarios.	Registro, edición, perfil, cambio de estatus, validación de administrador, claveUsuario y evidencias.
Integrante 3	VehicleService y dominio de vehículos.	Registro, búsqueda, edición, cambio de estatus, placa única, límite de 4 vehículos y validación para ParkingService.
Integrante 4	ParkingService, integración, Postman final y documento integrado.	Entrada, salida, espacios, cálculo de costo, validaciones integradas, colección Postman y evidencias finales.

Principio de trabajo en cadena

Cada integrante debe entregar algo funcional y consumible por el siguiente: endpoints estables, datos de prueba, JSON de ejemplo, errores esperados, capturas y explicación breve.

De	Para	Entrega necesaria
Integrante 1	Integrante 2	Token funcional, estructura de seguridad y usuario administrador válido.
Integrante 2	Integrante 3	Usuarios válidos, idUsuario, claveUsuario y validación de usuario activo.
Integrante 3	Integrante 4	Placas válidas, vehículos activos y validación de placa asociada al usuario.
Integrante 4	Equipo completo	Flujo final de entrada/salida, colección Postman y evidencias.
Todo el equipo	Documento final	Capturas, endpoints, casos de error y explicación de cada servicio.

5. Alertas de complicaciones

Riesgo	Por qué afecta	Cómo evitarlo
Confusión entre 3 dominios y 4 servicios	El documento habla de servicios de autenticación, usuarios, vehículos y estacionamiento, pero también de bases por dominio.	Presentar 4 servicios funcionales y explicar que AuthService y UserService pertenecen al dominio de usuarios.
ParkingService como cuello de botella	Depende de usuario, vehículo, cajones, movimientos, tiempo y costo.	Crear desde temprano endpoints de validación en UserService y VehicleService; probar con datos fijos.
Docker falla al final	Puertos, variables y conexiones suelen fallar aunque el código funcione localmente.	Dockerizar antes de cerrar documentación; probar Postman contra contenedores reales.
Postman desordenado	Una colección sin orden hace lenta y confusa la demo.	Ordenarla como flujo narrativo: login, usuario, vehículo, espacios, entrada y salida.
Documentación genérica	El profesor revisará claridad y evidencia del sistema real.	Usar tablas, diagramas y capturas reales; no copiar teoría innecesaria.

6. Checklist final antes de entregar

Área	Debe estar listo
------	------------------

AuthService	Login exitoso, token, bcrypt, usuario inactivo rechazado y error por credenciales incorrectas.
UserService	Registrar, editar, ver perfil, cambiar estatus, correo único, username único y claveUsuario generada.
VehicleService	Buscar, registrar, editar, cambiar estatus, placa única, máximo 4 vehículos y validación para ParkingService.
ParkingService	Consultar espacios, entrada, salida, ocupar/liberar cajón, máximo 2 vehículos y cálculo de costo.
Comunicación	ParkingService valida usuario y vehículo por API REST, no por acceso directo a otras bases.
Bases de datos	Usuarios en PostgreSQL; vehículos y estacionamiento en MySQL; datos de prueba cargados.
Docker	Servicios y bases levantan en contenedores; puertos y variables documentados.
Postman	Colección exportada, variables configuradas, casos exitosos y errores incluidos.
Documentación	Plantilla completa con diagramas, endpoints, tecnologías, pruebas, evidencias y conclusiones.
Exposición	Cada integrante sabe explicar su parte y el flujo completo.

Flujo mínimo que debe salir perfecto en la demo

Login -> token -> registrar usuario -> registrar vehículo -> consultar espacios -> registrar entrada -> verificar cajón ocupado -> registrar salida -> verificar cajón libre -> mostrar un caso de error.

Cierre estratégico

La estrategia más segura es construir únicamente lo demostrable y evaluable. **El proyecto debe verse como una línea de ensamblaje: autenticación valida, usuarios existen, vehículos pertenecen, estacionamiento registra movimientos y Postman demuestra todo.**

Conclusión

Si el equipo respeta la ruta, evita complejidad innecesaria y cada integrante entrega su parte funcional con evidencias, el proyecto queda alineado con los criterios principales para aspirar al 100%.